

阶段 5 学习文档：Tessendorf 海洋谱 + FFT 物理

0. 当前状态

你已经完成：

- Vulkan 基础对象封装；
- SwapChain / RenderPass / Framebuffer / Pipeline / CommandBuffer；
- UBO；
- Descriptor；
- Texture2D；
- 贴图四边形；
- MVP 旋转；
- resize / minimize / restore 不崩；
- validation layer 零 error。

现在正式进入：

阶段 5: Tessendorf 海洋谱 + FFT 物理

本阶段的关键词不是“渲染”，而是：

生成一套可验证的水面物理数据：高度、横向位移、斜率、法线、Jacobian。

本阶段暂时不要写 Vulkan 渲染代码。最多只允许新建一个纯 C++ 控制台程序，用来验证 FFT 海浪物理。

1. 阶段 5 的唯一目标

阶段 5 的最终产物不是屏幕上的水面，而是一个 CPU 端水面生成器：

输入：

- 网格分辨率 N
- 物理尺寸 L
- 风速 windSpeed
- 风向 windDirection
- 海浪强度 amplitude
- damping
- choppy 参数 lambda
- time

输出：

- displacement map: (dispX, height, dispZ, jacobian)
- normal/slope map: (slopeX, slopeZ, dDxdx, dDzdz)

这个模块以后会接到阶段 6：

阶段 5：
CPU 生成 displacement / normal 数据

阶段 6：
把这些数据上传成 Vulkan texture
顶点 shader 采样 displacement map
水面网格开始起伏

所以阶段 5 可以理解为：

先造出“水面运动引擎”，再谈“水面渲染”。

2. 现在最重要的边界

2.1 不要现在做的事

阶段 5 暂时不要做：

- 不要写 WaterSurfaceMesh；
- 不要写水面网格；
- 不要写顶点位移 shader；
- 不要写 Fresnel；
- 不要写折射；
- 不要写泡沫；
- 不要写天空；
- 不要调视觉参数；
- 不要追求“好看”。

这些是阶段 6、7、8 的内容。

2.2 现在必须做的事

阶段 5 只做：

- 理解频域海浪；
- 理解 Phillips spectrum；
- 理解随机初始频谱；
- 理解深水色散关系；
- 理解时间演化；
- 理解 IFFT；
- 理解 choppy wave；
- 理解斜率和法线；
- 用 CPU 程序输出可验证的数据。

一句话：

阶段 5 是“物理数据生成”，不是“水面画面制作”。

3. 总学习路线

本阶段按下面顺序学：

水面为什么可以用 height field 表示
↓
为什么大量正弦波可以叠加成海面
↓
为什么正弦波叠加可以放到频域
↓
Tessendorf 如何用 Phillips 谱描述风驱动海浪能量
↓
如何用高斯随机数生成初始频域振幅 $h_0(k)$
↓
如何用色散关系 $\omega(k)$ 推进时间
↓
如何用 IFFT 从频域回到空间域
↓
如何生成高度、横向位移、斜率、法线
↓
如何把结果输出为 PNG / PPM / CSV 验证
↓
通过阶段 5 验收后进入阶段 6

4. 阅读材料分层

你现在材料很多，不要平均用力。必须分成四层。

4.1 第一层：必须精读

这些材料直接服务于阶段 5。

A. 《从零手写水体渲染器 · 学习方案》

阅读重点：

- 学习铁律；
- 阶段 5 的目标；
- 阶段 5 的公式主线；
- 阶段 5 的实现函数；
- 阶段 5 的验收标准；
- 阶段 6 的接口要求。

阅读方式：

1. 先读阶段 5 全部内容。

2. 把里面列出的每个公式单独抄到笔记里。
3. 每个函数名旁边写一句话说明它做什么。
4. 不看源码，自己画出数据流图。

你要从这份文档里提炼出下面这条链：

Phillips spectrum
→ $h_0(k)$
→ $h(k,t)$
→ IFFT
→ height / displacement / slope / normal

B. Tessendorf 《Simulating Ocean Water》

这是阶段 5 的核心论文。

阅读三遍。

第一遍：读故事

目标不是推公式，而是回答：

- Tessendorf 为什么不做完整 Navier-Stokes?
- 为什么只模拟水面?
- 为什么使用 height field?
- 为什么海面可以看成大量波的叠加?
- 这个方法适合什么，不适合什么?

第一遍只要求建立直觉。

第二遍：读公式

重点公式：

$$h(x,t) = \sum h_{\sim}(k,t) \exp(i \cdot k \cdot x)$$

$$P_h(k) = \text{Phillips spectrum}$$

$$h_0(k) = \text{randomGaussian} \times \sqrt{P_h(k)}$$

$$\omega(k) = \sqrt{g \cdot |k|}$$

$$h(k,t) = h_0(k) \exp(i\omega t) + h_0^*(-k) \exp(-i\omega t)$$

你必须理解每个符号：

符号	含义
x	空间位置
k	波矢量
$h(x,t)$	空间域水面高度
$\tilde{h}(k,t)$	频域复振幅
$h_0(k)$	初始频域振幅
$P_h(k)$	Phillips 谱
$\omega(k)$	角频率
g	重力加速度

第三遍：转成代码

读完第三遍后，你应该能写出下面的伪代码：

```
ComputeWaveVectors();
ComputeGaussianRandom();
ComputeBaseSpectrum();
ComputeWaves(time);
RunIFFT();
BuildDisplacementAndNormalMaps();
```

4.2 第二层：基础理解材料

这些材料帮助你理解“为什么水面可以用二维高度场近似”。

C. 《波动方程导出与定解条件》

阅读重点：

- 什么是波动方程；
- 为什么波动方程是双曲型 PDE；
- 初始条件是什么；
- 边界条件是什么；
- 叠加原理为什么成立。

阶段 5 不要求你完整解 PDE，但你必须理解：

线性波可以叠加。

这是 Tessenrodt 海面的根基之一。

D. 《波动方程书籍章节》

阅读重点：

- 一维波动方程；
- Cauchy 问题；
- 叠加原理；
- 能量守恒；
- 波传播速度。

你不需要钻入全部证明。只要能回答：

- 为什么线性系统里多个波可以相加？
- 为什么波不是“整体移动”，而是“相位传播”？
- 为什么初始扰动会向外传播？

E. 《Fast Water Simulation Using Height Fields》

阅读重点：

- Height field water 是什么；
- 为什么游戏中常把 3D 水体降维成 2D height field；
- 为什么用两个数组 u 和 v 就能做一个简单水波；
- 有限差分波动方程和 FFT 海洋有什么区别。

你要特别理解这个对比：

有限差分高度场：
适合池塘、水坑、局部交互波纹。

Tessendorf FFT：
适合大海、远洋、风驱动的大范围周期性海面。

4.3 第三层：对比阅读材料

这些材料不是阶段 5 的实现核心，但能帮你建立水体技术地图。

F. Wave Particles 论文和 PPT

阅读目标：

- 明白它解决的是“物体交互波”；
- 明白它和 Tessendorf 的差别；
- 明白为什么 Tessendorf 海面虽然真实，但不天然适合双向物体交互。

只读这些问题：

- wave particle 是什么？
- 它为什么适合船、水箱、边界反射？
- 它和 height field 的关系是什么？
- 它是否替代 Tessendorf? 答案：不替代，它是交互波补充方案。

阶段 5 不实现 wave particles。

G. open_water.pdf

阅读重点：

- ambient waves;
- interactive waves;
- 大范围海面为什么通常分成“背景海浪”和“交互波”；
- Tessendorf 适合 ambient waves;
- 2D wave equation / FDM 适合交互波。

你要建立这个概念：

大海水面通常不是一个算法解决全部问题。

背景海浪：

Tessendorf / FFT / Gerstner / procedural waves

局部交互：

2D wave equation / wave particles / shallow water / particles

视觉增强：

foam / caustics / reflection / refraction / screen-space effects

H. hfFluid.pdf

阅读目标：

- 理解实时游戏为什么经常把三维流体降成二维高度场；
- 理解 height field 的优点和缺陷；
- 理解瀑布、翻卷浪、飞溅不能只靠 height field 表示；
- 理解粒子和高度场的混合思想。

阶段 5 不实现浅水方程，也不实现粒子。

4.4 第四层：暂时只浏览，不深入

这些材料后面再读。

I. An Efficient Method for Rendering Underwater Optical Effects

现在不精读。阶段 7 再读。

当前只知道它属于：

Shading / underwater optics / caustics

不是阶段 5 任务。

J. Sea of Thieves 技术分享

现在只浏览。

阅读目标：

- 知道工业项目里确实使用 FFT ocean；
- 知道还会额外加入 stylization、foam、scattering approximation；
- 知道真实游戏水体不是单纯 Tessendorf，而是多个技巧叠加。

阶段 5 不照着它实现。

K. Horizon Forbidden West Water

现在只浏览。

它属于更高级工业方案，特别是 breaking waves、baked simulation、复杂水体系统。现在不要被它带偏。

阶段 5 的目标不是做 Horizon 级别水体。

L. GDC / 插件横评 / 浮力文档

现在只作为方向感材料，不进入阶段 5 实现。

后面用途：

材料	适合阶段
浮力文档	阶段 9 或扩展阶段
插件横评	系统设计参考
GDC VFX	泡沫、波纹、特效参考
Horizon / Sea of Thieves	工业技术路线参考

5. 阶段 5 时间安排

建议按 12 天执行。不要压缩到 3 天，否则只会变成“抄公式”。

Day 0：整理阶段 4，建立阶段 5 文件夹

目标

把阶段 4 收尾，不让旧问题污染阶段 5。

要做

1. 确认 stage4_textured_quad 能稳定运行。
2. 确认 validation layer 零 error。
3. 在笔记里写清楚：
4. DescriptorSetLayout 是什么；
5. DescriptorPool 是什么；
6. DescriptorWriter 是什么；
7. Texture2D 的 image layout 转换流程；
8. UBO 每帧更新流程。
9. 新建目录：

```
docs/  
  stage5_tessendorf_notes.md  
  
experiments/  
  stage5_ocean_cpu/  
    main.cpp  
    CMakeLists.txt  
    OceanSpectrum.h  
    OceanSpectrum.cpp  
    ImageWriter.h  
    ImageWriter.cpp
```

禁止

不要把阶段 5 直接塞进 Vulkan 主程序。

Day 1：建立水体技术地图

阅读

- 你的水体技术分类图；
- 学习方案阶段 5、6、7；

- open_water 的 overview;
- hfFluid introduction。

目标

明白：

Shape 负责水面形状。
Shading 负责水面颜色和光学。
Meshing 负责如何铺水面网格。
Game solution 负责工程化系统。

阶段 5 只属于：

Shape / 统计谱方法 / FFT ocean

输出笔记

写一页：

```
# 水体技术分类与本阶段边界

## 本阶段做
- Tessendorf FFT ocean
- height / displacement / normal map

## 本阶段不做
- Fresnel
- foam
- caustics
- buoyancy
- underwater rendering
- interactive ripples
```

Day 2：复习波动方程和叠加原理

阅读

- 波动方程导出材料；
- 波动方程书籍章节；
- Fast Water Simulation Using Height Fields 前几页。

必须搞懂

1. 什么是高度场：

水面 = $y = h(x, z, t)$

1. 什么是线性叠加：

如果 h_1 是一个波， h_2 是一个波，
那么 $h = h_1 + h_2$ 仍然可以表示组合波形。

1. 为什么海面可以近似成很多波的叠加。

输出笔记

写出：

一个正弦波：

$$h(x, t) = A \cos(kx - \omega t + \phi)$$

多个波：

$$h(x, t) = \sum A_i \cos(k_i x - \omega_i t + \phi_i)$$

不要求

不要求完整掌握浅水方程。

Day 3：复数、傅里叶、DFT、IFFT

目标

理解为什么阶段 5 要用复数数组。

必须掌握

$$e^{i\theta} = \cos\theta + i \sin\theta$$

频域：

每个 k 存一个复数，表示某个方向、某个波长的波的振幅和相位。

空间域：

每个网格点存一个真实高度。

手算练习

用 4 个采样点做一个极小 DFT / IDFT 例子。

不需要追求速度，只要理解：

频域的一组复数
通过 IFFT
变成空间网格上的高度

输出笔记

写出：

我对 DFT / IFFT 的理解

频域数组不是图像。
它是“一堆波的配方”。

IFFT 不是魔法。
它只是把所有波加回空间位置。

Day 4: Tessendorf 第一遍——只读故事

阅读

Tessendorf 《Simulating Ocean Water》第一遍。

目标

只回答大问题：

1. 为什么不做完整 3D fluid?
2. 为什么只做 surface?
3. 为什么使用 height field?
4. 为什么大海适合统计模型?
5. 为什么风速和风向能决定波谱?
6. 为什么 FFT ocean 是周期性 tile?

输出笔记

写一页：

Tessendorf 方法解决什么问题

它用统计海洋模型生成大范围开放水面。
它不是物体交互水波求解器。
它适合 ocean ambient waves。
它的结果是周期性 height field。

Day 5: Tessendorf 第二遍——拆公式

阅读

重新读 Tessendorf 的频谱公式。

必须拆清楚

1. 波矢量 k

$$k = (k_x, k_z)$$

它表示：

- 波的方向；
- 波长；
- 空间频率。

2. Phillips 谱

它控制每个 k 上有多少能量。

你要理解每一项大概作用：

项	作用
amplitude	总体浪高
wind direction	波是否沿风向增强
wind speed	决定大波尺度
	k
damping	抑制过小波长

3. $h_0(k)$

$$h_0(k) = \text{GaussianRandom} \times \text{sqrt}(\text{Phillips}(k))$$

意思：

Phillips 决定能量大小。
GaussianRandom 提供随机性。

4. 时间演化

$$h(k, t) = h_0(k)e^{i\omega t} + h_0^*(-k)e^{-i\omega t}$$

意思：

频域中的每个波按自己的角频率旋转相位。

输出笔记

必须写出公式和中文解释。

Day 6: 先写朴素 DFT，不接 FFTW

目标

写一个最小 CPU 程序，不求快，只求对。

实现内容

```
struct Complex {  
    float real;  
    float imag;  
};
```

或者直接用：

```
std::complex<float>
```

实现：

```
NaiveIDFT2D(inputFreq, outputSpatial, N);
```

先用 $N = 8$ 或 $N = 16$ 。

验证

构造一个简单频域：

只有一个 k 有非零值

IFFT 后应该得到规则正弦波。

输出

输出 CSV:

```
x,z,height
```

或者输出灰度 PPM:

```
height_000.ppm
```

禁止

这一天不要接 FFTW。

先把频域到空间域的概念跑通。

Day 7: 实现 k-grid、Phillips、h0(k)

目标

实现初始海洋频谱。

函数

```
void ComputeWaveVectors();  
float PhillipsSpectrum(glm::vec2 k);  
std::complex<float> GenerateH0(glm::vec2 k);  
void ComputeBaseSpectrum();
```

注意点

必须处理:

```
k = 0
```

否则会除零。

必须固定随机种子:

```
std::mt19937 rng(1234);
```

否则每次输出不一致, 不利于调试。

输出

输出几张调试图：

```
phillips_spectrum.ppm  
h0_magnitude.ppm
```

验收

你应该能看出：

- 风向相关性；
- 某些频率能量高；
- $k=0$ 附近没有爆炸值；
- 没有 NaN；
- 没有 Inf。

Day 8: 实现 ComputeWaves(t)

目标

把初始频谱推进到任意时间。

函数

```
void ComputeWaves(float time);
```

数据流

```
h0(k)  
h0*(-k)  
 $\omega(k)$   
time  
    ↓  
h(k,t)  
    ↓  
IFFT  
    ↓  
height(x,z,t)
```

先用 NaiveIDFT

这一天仍然可以用朴素 DFT。
N 不要太大，用 $N = 16$ 或 $N = 32$ 。

输出

输出多帧：

```
height_t000.ppm  
height_t001.ppm  
height_t002.ppm
```

验收

连续帧应该是“波在动”，不是随机噪声闪烁。

Day 9：接入 FFTW

目标

把 NaiveIDFT 替换成 FFTW。

要理解

```
FFTW plan = 预先规划好的 FFT 执行方案  
FFTW_BACKWARD = 频域到空间域  
fftwf_execute = 执行变换
```

实现

```
SetupFFTW();  
ExecuteIFFT();  
DestroyFFTW();
```

对比测试

同一个输入：

```
NaiveIDFT 输出  
FFTW 输出
```

比较误差。

允许有浮点误差，但不允许完全不同。

注意

检查 FFTW 是否需要你手动除以：

$N * N$

很多 FFT 库不会自动归一化。

Day 10: 加入 choppy displacement、slope、normal

目标

从单纯高度场升级为完整水面数据。

需要生成

```
height
dispX
dispZ
slopeX
slopeZ
dDxdx
dDzdz
jacobian
```

理解

高度：

$h(x, z)$

斜率：

```
 $\partial h / \partial x$ 
 $\partial h / \partial z$ 
```

法线近似：

```
normal = normalize(vec3(-slopeX, 1, -slopeZ))
```

横向位移：

让波峰更尖，水面不只是上下动。

Jacobian：

衡量局部压缩程度，可用于后续泡沫判断。

阶段 5 只生成 Jacobian，不做泡沫。

Day 11：验证与调试

数值检查

每一帧检查：

```
height min / max
dispX min / max
dispZ min / max
slopeX min / max
slopeZ min / max
normal length
NaN count
Inf count
```

必须满足

```
没有 NaN
没有 Inf
height 可归一化到 [-1,1]
normal 长度接近 1
连续帧不闪烁
固定随机种子下结果可复现
```

输出图片

至少输出：

```
height_t000.ppm
height_t050.ppm
height_t100.ppm
normal_t000.ppm
displacement_x_t000.ppm
displacement_z_t000.ppm
```

视觉判断

正确的图应该是：

- 连续起伏；
- 没有孤立突刺；

- 没有棋盘格闪烁；
- 波纹方向和风向有关；
- 修改风速后波尺度变化合理。

Day 12: 整理成正式模块

目标

把实验程序整理成未来可以接入渲染器的类。

建议类结构

```
class WTessendorf {
public:
    struct Settings {
        uint32_t N;
        float L;
        float amplitude;
        float windSpeed;
        glm::vec2 windDirection;
        float damping;
        float lambda;
    };

    explicit WTessendorf(const Settings& settings);

    void ComputeWaves(float time);

    const std::vector<glm::vec4>& GetDisplacements() const;
    const std::vector<glm::vec4>& GetNormals() const;

private:
    void ComputeWaveVectors();
    void ComputeGaussianRandomArray();
    void ComputeBaseWaveHeightField();
    void SetupFFTW();
    void ExecuteIFFT();
};
```

不要做

不要现在上传 Vulkan texture。
不要现在写 WaterSurfaceMesh。
这属于阶段 6。

6. 到什么时候继续写渲染器？

严格标准：

只有当阶段 5 的 CPU 海浪模块通过验收后，才继续写 Vulkan 渲染器。

具体来说，满足下面全部条件才进入阶段 6。

6.1 数据验收

你必须已经能输出：

```
displacement map
normal map
height image
normal image
```

并且：

```
无 NaN
无 Inf
高度范围合理
法线长度合理
多帧连续
风向变化有效
风速变化有效
```

6.2 理解验收

你必须能不看源码回答：

1. 为什么海面可以看成正弦波叠加？
2. k 向量是什么？
3. Phillips 谱每一项大概在控制什么？
4. $h_0(k)$ 为什么要乘高斯随机数？
5. 为什么 $h(k,t)$ 要包含 $h_0(k)$ 和 $h_0^*(-k)$ 两项？
6. IFFT 的输入是什么，输出是什么？
7. choppy displacement 为什么能让浪尖更尖？
8. slope 和 normal 是怎么来的？
9. $k=0$ 为什么必须特殊处理？
10. FFTW 的 backward transform 为什么通常需要归一化？

6.3 代码验收

你的阶段 5 代码应该满足：

可以独立编译为 console 程序
不依赖 Vulkan
可以固定参数输出图片
可以调整 $N = 32 / 64 / 128$
可以稳定运行多帧
核心函数命名清晰
没有把调参写死在算法内部

满足这些后，才能开始阶段 6。

7. 阶段 6 开始后要做什么

阶段 6 的入口不是重新研究物理，而是把阶段 5 的输出接入 Vulkan。

阶段 6 的第一步应该是：

```
CreateGridVertices()  
CreateGridIndices()
```

先在屏幕上画一片静态平面网格。

第二步：

把阶段 5 的 displacement map 上传成 R32G32B32A32_SFLOAT texture

第三步：

vertex shader 采样 displacement map
修改顶点位置

第四步：

上传 normal/slope map
给后续 shading 准备

所以阶段 6 的数据流是：

```
WSTessendorf::ComputeWaves(t)  
↓  
CPU displacement / normal arrays  
↓  
staging buffer  
↓
```

```
VkImage displacementTexture / normalTexture
↓
DescriptorSet binding2 / binding3
↓
Vertex shader sample
↓
水面网格起伏
```

8. 阶段 5 常见坑

坑 1：太早接 Vulkan

错误做法：

还没验证 FFT 数据，就开始写 Vulkan texture 上传。

后果：

你不知道画面错误是 FFT 错，还是 Vulkan 错，还是 shader 错。

正确做法：

先用 CPU 输出图片验证物理。

坑 2：k=0 没处理

很多公式里有：

```
1 / |k|
1 / |k|^2
1 / |k|^4
```

如果 k=0，会直接爆炸。

正确做法：

```
if (length(k) < epsilon) return 0;
```

坑 3：忘记 FFT 归一化

有些 FFT 库执行 IFFT 后不会自动除以：

$$N * N$$

如果忘了除，浪高会大得离谱。

坑 4：复共轭关系错

如果频域构造不满足实数场要求，IFFT 后会产生不合理的虚部。

最终高度场应该是实数。

如果虚部很大，说明频谱对称或时间演化有问题。

坑 5：频谱中心化符号翻转漏掉

Tessendorf 实现中常见：

$$(-1)^{(m+n)}$$

这个用于频谱中心化 / 空间排列修正。
漏掉后可能出现错位、闪烁或波形不对。

坑 6：风向没有归一化

风向必须是单位向量。

错误：

```
wind = glm::vec2(10, 3);
```

正确：

```
wind = normalize(glm::vec2(10, 3));
```

坑 7：随机数每帧重算

$h_0(k)$ 是初始频谱。

它应该初始化时生成一次。

错误做法：

每帧重新生成 $h_0(k)$

后果：

水面每帧随机闪烁。

正确做法：

$h_0(k)$ 固定，只有相位随 time 演化。

9. 每日笔记模板

每天学习后都写下面四项：

Day X 学习记录

今天读了什么

今天真正理解了什么

今天实现了什么

还有什么不懂

阶段 5 期间尤其要记录：

公式解释

公式：

...

每个符号：

...

它在代码里对应：

...

不要只写“我读完了”。

必须写“我能不能把公式变成代码”。

10. 阶段 5 最终验收文档

阶段 5 结束时，你应该写一份 `stage5_acceptance.md`：

```
# 阶段 5 验收报告

## 1. 实现了哪些函数

## 2. 输入参数

## 3. 输出数据

## 4. 数值检查

## 5. 输出图片

## 6. 我对 Tessendorf 数据流的理解

## 7. 进入阶段 6 前仍然要注意的问题
```

只有写完这份验收报告，才正式进入阶段 6。

11. 最终执行结论

当前阶段的正确路线是：

```
先读主学习方案阶段 5
↓
读 Tessendorf 三遍
↓
补波动方程和高度场基础
↓
写朴素 DFT 验证
↓
写 Phillips / h0 / h(k,t)
↓
接 FFTW
↓
输出 displacement / normal
↓
通过 CPU 验收
↓
进入阶段 6 继续写 Vulkan 水体渲染器
```

最重要的一句话：

阶段 5 不追求画面，追求数据正确。
阶段 6 才把正确的数据画到屏幕上。